And fill out the CreateContacts migration with the following code:

phone_app/priv/repo/migrations/20230507184731_create_contacts.exs

```
defmodule PhoneApp.Repo.Migrations.CreateContacts do
  use Ecto.Migration

  def change do
    create table(:contacts) do
      add :phone_number, :text, null: false
      add :name, :text

      timestamps(type: :utc_datetime_usec)
    end

    create unique_index(:contacts, [:phone_number])
  end
end
```

A migration consists of either a single change/0 function or the combination of up/0 and down/0 functions. Ecto runs the correct function based on whether we are migrating forward or rolling back a migration. Similar to ActiveRecord migrations, some operations are "reversible" and others are one-way. For example, create table can be reversed by dropping the table. But if we were to modify a column, then it would only be reversible if we provided the function with the original column definition.

The migration files are fairly self-explanatory. The great part of the migration DSL is that it's easy to read—it's inspired by the ActiveRecord migration format. All of the fields in our schemas are represented in the migration, with a PostgreSQL type that corresponds to the Ecto schema field type.

Finally, run your migrations with the command line migrator:

```
$ mix ecto.migrate
15:06:04.213 [info] == Running...CreateSmsMessages.change/0 forward
15:06:04.217 [info] create table sms_messages
15:06:04.228 [info] create index sms_messages_contact_id_index
15:06:04.231 [info] create index sms_messages_message_sid_index
15:06:04.236 [info] == Migrated 20230507184727 in 0.0s

15:06:04.274 [info] == Running...CreateContacts.change/0 forward
15:06:04.274 [info] create table contacts
15:06:04.279 [info] create index contacts_phone_number_index
15:06:04.281 [info] == Migrated 20230507184731 in 0.0s
```

We now have our schema and corresponding (empty) database tables. Before we move on, we're going to look quickly at the dangers of migrations.